

ADR 001: Implementación de Provider Strategy Pattern para Sistema Multicanal

1. Estado

Aceptado

2. Contexto

El sistema requiere enviar notificaciones a través de diversos canales (Email, SMS, Push) con comportamientos distintos:

- El canal de **Email** debe permitir integraciones reales (ej. Resend/Nodemailer).
- Los canales de **SMS y Push** deben ser simulados (Mocks) para este MVP.
- Se requiere soporte multi-idioma y una lógica de reintentos en caso de fallo del proveedor.

Implementar esta lógica mediante estructuras condicionales simples (if/else o switch) dentro de los controladores de la API generaría un código acoplado, difícil de testear y propenso a errores al intentar escalar a nuevos canales o proveedores.

3. Decisión

He decidido implementar el **Patrón de Diseño Strategy** (también conocido como **Provider Pattern** en este contexto).

La arquitectura se divide en tres capas principales:

1. **Capa de Orquestación (Service):** Gestiona el flujo de negocio, la persistencia de logs y la lógica de reintentos.
2. **Abstracción (Interface):** Una interfaz común `INotificationProvider` que define el contrato `send(data)`.
3. **Estrategias Concretas (Providers):** Clases independientes para cada canal (`EmailProvider`, `SMSMockProvider`, `PushMockProvider`).

4. Razones de la Elección

1. **Principio de Responsabilidad Única (SRP):** Cada proveedor se encarga exclusivamente de la comunicación con su respectivo servicio externo.
2. **Principio Open/Closed (SOLID):** Podemos añadir nuevos canales (ej. WhatsApp) creando una nueva clase sin modificar el código existente en el servicio de orquestación.
3. **Testabilidad y Simulación:** Permite intercambiar fácilmente el proveedor real por un Twilio-Mock durante el desarrollo y las pruebas, asegurando que la lógica de reintentos funcione sin incurrir en costos de API.

4. **Resiliencia:** Facilita la implementación de un sistema de estados centralizado (Pending, Retried, Sent, Failed) que es independiente de la tecnología de envío.

5. Sacrificios (Trade-offs)

1. **Complejidad Inicial:** Requiere la creación de más archivos e interfaces que una solución monolítica simple, lo que aumenta la curva de aprendizaje inicial del proyecto.
2. **Indirección:** El flujo de datos no es lineal; la lógica de envío se delega, lo que requiere un seguimiento más cuidadoso durante el debugging manual.

6. Consecuencias

- **Positivas:** El sistema es altamente mantenible. Si un proveedor de Email cambia sus políticas o SDK, el impacto se limita exclusivamente a su clase correspondiente.
 - **Negativas:** El manejo de errores debe ser estandarizado en todos los proveedores para que la capa de orquestación pueda interpretar los fallos y ejecutar los reintentos correctamente.
-